SonicMAE: Three-Person Presentation Script & Code Defense Guide

Course: Deep Learning / Industrial Signal Processing
Project Title: SonicMAE: Self-Supervised Masked Autoencoder with Phase-Scrambled Contrastive Representation
Team Size: 3 Members (Complete Individual Work Breakdown & Script)
GitHub Repository: https://github.com/Amrutha-Tsavatapalli/SONIC_MAE

1. Team Responsibility Matrix

Role	Primary Module	Key Deliverables	Key Code Files to Open

Person 1	Data Ingestion, DSP Pipeline & Scientific Novelty (CSPS)	Dataset EDA, 16 kHz Log-Mel extraction, Conjugate-Symmetric Phase Scrambling math & proof	`src/sonicmae/audio.py`
			`src/sonicmae/augment.py`
			`tests/test_signal_pipeline.py`

Person 2	Model Architectures, Self-Supervised Training & Transfer Learning	SonicMAE, SonicConformerMAE, TransferAcousticNet, 25% masking, InfoNCE loss, Ablation grid	`src/sonicmae/model.py`
			`src/sonicmae/conformer.py`
			`src/sonicmae/transfer.py`
			`scripts/train_advanced_architectures.py`

Person 3	Scoring Engine, Explainable Sonification, Edge SLA & UI	Dual-stream scoring, Multi-Centroid bank, Griffin-Lim sonification, INT8 quantization, Streamlit UI	`src/sonicmae/scoring.py`
			`src/sonicmae/sonify.py`
			`scripts/benchmark_edge.py`
			`app.py`

2. Person 1: Signal Processing, Data Pipeline & Scientific Novelty (CSPS)

> **Person 1 Assignment Overview**
>
> You open the presentation, state the industrial problem, introduce the MIMII DUE pump dataset, explain the Log-Mel DSP transformation, and present our **Core Scientific Novelty: Conjugate-Symmetric Phase Scrambling (CSPS)**.

2.1 Spoken Presentation Script

> **SPOKEN SCRIPT — PERSON 1 (OPENING & PROBLEM MOTIVATION)**
>
> "Good morning, respected mam and evaluators. Today, our team presents **SonicMAE**, an unsupervised self-supervised deep learning system for industrial acoustic anomaly detection, benchmarked on industrial centrifugal water pumps from the MIMII DUE dataset.
>
> In an operating factory, water pumps run continuously. Catastrophic mechanical breakdown causes unplanned downtime costing millions of dollars and creates major safety hazards. However, in real-world condition monitoring, **we face severe class imbalance**: 99.9% of audio is normal operational sound. Fault audio—such as impeller cavitation or bearing race spalling—is rare, hazardous, and cannot be collected at scale for supervised training.
>
> Therefore, our goal is 100% unsupervised: the model must learn what healthy machine physics sounds like purely from normal operational recordings."

SPOKEN SCRIPT — PERSON 1 (DSP PIPELINE & SCIENTIFIC NOVELTY)

"To process continuous machinery audio, I developed our DSP feature extraction pipeline in `src/sonicmae/audio.py`. We downsample audio to 16 kHz mono, apply peak normalization, and compute an STFT with an FFT window of 1024 and hop size of 512. This is converted into a 64-channel log-Mel spectrogram spanning 320 frames, corresponding to a 10-second temporal window.

Now, mam, if you look at standard audio self-supervision in speech or music, researchers use pitch-shifting or speed-tuning. **In industrial machinery, standard augmentations fail completely because pitch-shifting changes the physical motor rotational speed (f_motor) and shifts harmonic poles (k * f_0), violating physical machine laws.**

To solve this, our project introduces our core scientific novelty: **Conjugate-Symmetric Phase Scrambling (CSPS)** in `src/sonicmae/augment.py`. We perturb only the phase angles by delta_theta ~ Uniform(-pi * lambda, pi * lambda) while strictly enforcing complex conjugate reflection across the Nyquist frequency. Because |exp(j * delta_theta)| = 1, **100% of the magnitude spectrum and physical harmonics remain identical**, while the model learns via contrastive alignment to be completely immune to microphone placement, sensor angles, and room reverberation."

2.2 Exact Code Walkthrough to Show "Mam"

Step 1: Open `src/sonicmae/audio.py`

Lines to highlight: `log_mel()` and `load_wav()`.
What to explain to mam: "Mam, look at lines 15-28. Here is where we compute the STFT using `librosa.feature.melspectrogram` with 64 Mel bins, power=2.0, and hop length 512. Then in line 25, we take `np.log(mel + 1e-6)` to compress acoustic dynamics and use `fixed_frames()` to ensure every recording has exactly 320 frames (10 seconds) for batch tensor operations."

Step 2: Open `src/sonicmae/augment.py`

Lines to highlight: `phase_scramble()`.
What to explain to mam: "Mam, this is our novel CSPS algorithm. In line 18, we compute `np.fft.rfft(signal)` to get the complex spectrum. In line 21, we add random phase rotation bounded by lambda = 0.20. In line 24, `np.fft.irfft()` automatically enforces complex conjugate symmetry X[N-k] = X[k]*, ensuring the reconstructed time signal is purely real-valued with exact magnitude spectrum preservation."

2.3 Anticipated Q&A for Person 1

Q1: "Why 64 Mel bins and not standard MFCCs?"
Answer: "MFCCs apply a Discrete Cosine Transform that discards spatial phase relationships and high-frequency harmonic structures. Log-Mel spectrograms preserve 2D time-frequency locality, allowing convolutional and attention layers to learn spatial acoustic textures."

Q2: "What happens if you do NOT enforce complex conjugate symmetry in phase scrambling?"
Answer: "If complex conjugate symmetry is broken, the inverse Fourier transform produces an imaginary signal with non-zero complex values. Machinery vibration sensors record purely real physical acoustic pressures; CSPS guarantees a valid physical signal."

3. Person 2: Deep Learning Architectures, Self-Supervision & Transfer Learning

> **Person 2 Assignment Overview**
>
> You present the model topologies: the baseline `SonicMAE`, our customized `SonicConformerMAE`, and the `TransferAcousticNet` (MobileNetV2). You explain the 25% masking strategy, the InfoNCE contrastive alignment loss, and the ablation results.

3.1 Spoken Presentation Script

> **SPOKEN SCRIPT — PERSON 2 (ARCHITECTURAL INNOVATION & CONFORMER)**
>
> "Thank you, [Person 1]. Respected mam, once the log-Mel representations are generated, the core deep learning question is: **What architecture can detect both subtle high-frequency friction and long-range periodic motor rotation cycles?**
>
> Standard CNN autoencoders fail because convolutional pooling smooths over high-frequency transient clicks, causing over-reconstruction (only 0.5702 ROC-AUC). Meanwhile, pure Vision Transformers like AudioMAE require massive 2-million-sample pretraining and use 75% masking, which destroys continuous audio harmonics.
>
> To solve this, I designed our customized architecture: **SonicConformerMAE** in `src/sonicmae/conformer.py`. We downsample the spectrogram through a convolutional stem and pass it through stacked Conformer blocks. Each block pairs **Depthwise Separable 1D Convolutions**—which isolate localized bearing flaking and friction transients—with **Multi-Head Self-Attention**, which learns harmonic synchronization across full 10-second pump rotational periods.
>
> Furthermore, we integrate a **Macaron-style SwiGLU Feed-Forward Network** before and after attention to stabilize gradient propagation."

SPOKEN SCRIPT — PERSON 2 (SELF-SUPERVISED LOSS & TRANSFER LEARNING)

"Our training objective combines two complementary self-supervised tasks:
1. **Masked Patch Reconstruction:** We apply a tailored 25% time-frequency patch mask. The decoder must reconstruct the missing spectral patches from surrounding context.
2. **InfoNCE Contrastive Invariance:** We pass both the masked view and the CSPS phase-scrambled view through the encoder and align their 128-dimensional latent projections using InfoNCE loss.

In addition, to benchmark against foundation model paradigms, I implemented **TransferAcousticNet** in `src/sonicmae/transfer.py`, adapting ImageNet pre-trained MobileNetV2 with a 1-channel adapter and InfoNCE projection head.

As shown in our benchmark comparison table in `final_model_comparison.csv`, our customized **SonicConformerMAE achieves 97.14% Precision and 0.6538 F1-score**—the highest among all deep learning models on the MIMII DUE benchmark."

3.2 Exact Code Walkthrough to Show "Mam"

Step 1: Open `src/sonicmae/conformer.py`

Lines to highlight: `ConformerBlock` and `SonicConformerMAE`.
What to explain to mam: "Mam, look at lines 60-85 in `conformer.py`. Here is our `ConformerBlock`. Notice the Macaron structure: first `self.ffn1` with SwiGLU activation, then `self.self_attn` with 4 multi-head attention heads, then `self.conv_module` with depthwise separable convolution, followed by `self.ffn2`. In lines 110-130, you can see our `mask_patches()` function applying our 25% patch mask."

Step 2: Open `scripts/train_advanced_architectures.py`

Lines to highlight: `infonce_loss()` and joint loss computation.
What to explain to mam: "Mam, look at lines 28-36. Here is our InfoNCE contrastive loss: `sim = torch.mm(z_a, z_b.t()) / temperature`. And in lines 65-72, you can see our total loss function: L_total = L_MSE + 0.20 * L_InfoNCE, jointly training reconstruction and latent invariance."

3.3 Anticipated Q&A for Person 2

Q1: "Why 25% masking ratio and not 75% like Vision MAE?"
Answer: "Vision MAE masks 75% because natural images have heavy spatial redundancy (e.g. sky, grass). Audio log-Mel spectrograms have continuous harmonic lines across time; masking 75% destroys harmonic continuity. Our ablation study proved 25% achieves 0.786 AUC, whereas 50% drops to 0.718."

Q2: "What is the advantage of SwiGLU over standard ReLU in the feed-forward network?"
Answer: "SwiGLU combines Swish gating with bilinear multiplicative units: Swish(W1 x) * W2 x. It provides

smooth non-linear gating and prevents dying-neuron phenomena, leading to faster training convergence."

4. Person 3: Scoring Engine, Explainable Sonification, Edge SLA & UI

> **Person 3 Assignment Overview**
>
> You present the dual-stream anomaly scoring formulation, the Multi-Centroid Memory Bank ($K=5$), the 95th percentile validation threshold calibration, the Griffin-Lim acoustic residual resynthesis ("Hear the Defect"), INT8 edge quantization, and lead the live Streamlit dashboard demonstration.

4.1 Spoken Presentation Script

> **SPOKEN SCRIPT — PERSON 3 (SCORING ENGINE & THRESHOLD CALIBRATION)**
>
> "Thank you, [Person 2]. Respected mam, once the model outputs a reconstructed spectrogram X^ and a latent embedding vector z, the operational question is: **How do we convert these outputs into a statistically robust, false-alarm-free decision?**
>
> I developed our **Dual-Stream Anomaly Scoring Engine** in `src/sonicmae/scoring.py`. We combine two normalized streams:
> 1. **Local Spectral Error:** Normalized MSE between input and reconstructed spectrograms, capturing high-frequency transient friction and clicks.
> 2. **Multi-Centroid Prototype Distance:** We fit a K=5 cluster centroid memory bank using K-Means on normal training embeddings. Using softmax temperature weighting (tau = 1.0), the model calculates the soft distance to healthy prototypes, allowing it to dynamically handle multi-modal pump operation under variable speeds and loads without false alarms.
>
> Crucially, mam, our decision threshold is mathematically calibrated at the **95th percentile of unseen healthy validation recordings (T = 2.510)**. This mathematically guarantees that the false positive alarm rate on healthy operational machinery cannot exceed 5%."

SPOKEN SCRIPT — PERSON 3 (EXPLAINABLE SONIFICATION, EDGE SLA & LIVE DEMO)

"To make our deep learning system actionable for factory technicians, I implemented three major features:

1. **Audible Residual Sonification ('Hear the Defect'):** In `src/sonicmae/sonify.py`, we take the positive spectral reconstruction error max(X - X^, 0) and invert it back to time-domain audio using the Griffin-Lim algorithm. This cancels out the steady normal pump droning, allowing technicians to **audibly hear just the metallic clicks, bearing wear, and cavitation bubbles**.

2. **Edge Quantization & Latency SLA:** In `scripts/benchmark_edge.py`, we applied PyTorch Dynamic INT8 quantization. On a standard CPU, inference takes only **5.26 ms per 10-second audio stream (190 streams/second)**, passing the sub-50 ms real-time edge gateway SLA with 0.999965 cosine similarity.

3. **Live Interactive Dashboard (`app.py`):** Let me now demonstrate our live system on screen: our interactive Pump Digital Twin with glowing component alarms, our live factory noise and phase stress-tester, our 4-pump fleet central control, and our one-click ISO-10816 inspection certificate generator."

4.2 Exact Code Walkthrough to Show "Mam"

Step 1: Open `src/sonicmae/scoring.py`

Lines to highlight: `fit_multi_centroid_reference()` and `multi_centroid_anomaly_score()`.
What to explain to mam: "Mam, look at lines 63-108. Here is where we fit K=5 cluster centers using K-Means. Then in line 90, we calculate softmax temperature distance: `weights = np.exp(-dists / temperature)` and return the unified score combining normalized reconstruction error and prototype distance."

Step 2: Open `src/sonicmae/sonify.py`

Lines to highlight: `resynthesize_residual()`.
What to explain to mam: "Mam, look at lines 15-45. Here we extract the positive spectral residual: `residual_spec = np.maximum(raw_spec - recon_spec, 0.0)` and invert it using `librosa.feature.inverse.mel_to_audio()` with Griffin-Lim. This allows us to convert the neural network's error directly into a playable WAV file on the dashboard."

Step 3: Open Browser at `http://localhost:8501` (Live App Demo)

What to click and show to mam:

1. Select `Abnormal Pump (pump_abnormal_recording.wav)`.
2. Play the original sound, then play the 🎧 **Hear the Extracted Defect** audio player. Point out: *"Mam, listen to how the motor hum is gone and you hear only the isolated friction clicks!"*
3. Switch to the ⚙️ **Digital Twin** tab and show the Bearing Housing glowing red.
4. Switch to the 🎛️ **Live Stress-Test** tab: drag noise down to 5 dB and phase jitter up to 0.5 to show SonicMAE staying robust while CNN-AE triggers false alarms.
5. Click 📥 **Download ISO-10816 Maintenance Certificate** to show the printable plant report.

4.3 Anticipated Q&A for Person 3

Q1: "Why calibrate at the 95th percentile instead of 99th or 90th?"
Answer: "In industrial machinery maintenance, 95th percentile calibration establishes an operational False Positive Rate of 5%. The 99th percentile misses early bearing degradation (low recall), while 90th causes alarm fatigue (10% false alarms). The 95th percentile achieves 97.14% precision in our benchmark."

Q2: "How does Griffin-Lim reconstruct audio from only Mel magnitude without phase?"
Answer: "Griffin-Lim is an iterative algorithm that alternates between computing the STFT and iSTFT, projecting estimated phase angles to enforce consistency with the target magnitude spectrum. Within 32 iterations, it produces clear, listenable audio."

5. Master Q&A Defense Reference (For All 3 Members)

Question Asked by Evaluator / Mam	Primary Responder	Winning Academic Response
"What is the novelty of your project?"	Person 1 & Person 2	"Our novelty lies in two domain-specific adaptations: 1) **Conjugate-Symmetric Phase Scrambling (CSPS)**, which preserves 100% of motor rotational harmonics while achieving phase invariance; and 2) **Low-Ratio (25%) Continuous-Spectral Masked Autoencoding with Conformer Attention**, overcoming the 75% vision MAE mismatch."
"Why is your ROC-AUC 0.78-0.85 while supervised classifiers claim 90%+?"	Person 2	"Supervised classifiers require labeled anomalous training audio. In industrial manufacturing, machine failure sounds are rare and hazardous to collect. Our model is **100% unsupervised**, trained strictly on healthy recordings, and achieves **97.14% Precision** at the 95th percentile operating point."
"Why did standard CNN Autoencoder perform so poorly (0.5702 AUC)?"	Person 2 & Person 3	"Standard CNN autoencoders optimize unmasked MSE across all pixels. Convolutional layers smooth out high-frequency transients, easily reconstructing anomalous clicks as background noise. SonicMAE forces context-aware patch reconstruction and latent prototype alignment, preventing over-reconstruction."
"Can this run on edge micro-controllers like Raspberry Pi?"	Person 3	"Yes, mam. We quantized the model to INT8 (checkpoint size 868 KB). Inference takes only **5.26 ms per 10-second audio stream** on a standard CPU, easily satisfying the sub-50 ms real-time industrial edge gateway SLA."